



Sistemas de Computación

Trabajo Practico N°4

Módulos de Kernel

Profesor:

Javier Alejandro Jorge

Grupo Mr.Robot

Ferraris, Gianluca
Gallone, Francesco
Marredo, Ezequiel

21/5/2026

Índice

Introducción.....	3
Objetivos.....	4
Preparación del entorno.....	5
Desafío 1.....	8
Desafío 2.....	9
User Space vs. Kernel Space.....	10
El Espacio de Usuario (User Space).....	10
El Espacio del Núcleo (Kernel Space).....	10
Organización de los Datos y Memoria.....	10
Controladores y el Directorio /dev.....	10
Algunos ejemplos de dispositivos con drivers pueden ser:.....	11
• Teclado.....	11
• Mouse.....	11
• Disco.....	11
• USB.....	11
• Audio.....	11
• GPU.....	11
Directorio /dev.....	11
Preguntas del trabajo práctico.....	12
¿Qué diferencias se pueden observar entre los dos modinfo ?.....	12
¿Qué drivers/modulos estan cargados en sus propias pc?.....	12
¿Cuales módulos no están cargados pero están disponibles?.....	13
¿Qué pasa cuando el driver de un dispositivo no está disponible?.....	14
Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte.....	14
¿Qué diferencia existe entre un módulo y un programa?.....	15
¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y como lo hace un programa?.....	16
Firmado de un módulo.....	17
¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mí?.....	19
¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?.....	19
¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?.....	20
¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?.....	20

Introducción

La arquitectura del kernel de Linux destaca por su diseño modular, el cual facilita la inserción y remoción de bloques de código —conocidos como módulos— durante el tiempo de ejecución, eliminando la necesidad de reiniciar el equipo. Esta capacidad de extensión dinámica es fundamental para la implementación de controladores de dispositivos (*drivers*). El presente trabajo profundiza en el ciclo de vida de estos módulos (desarrollo, compilación y gestión), analizando además la dicotomía entre el espacio de usuario y el de kernel, la interacción mediante el sistema de archivos `/proc`, el uso de la API de archivos y los protocolos de seguridad basados en firmas digitales.

Objetivos

Analizar la arquitectura interna del kernel de Linux mediante el diseño e implementación de módulos dinámicos, profundizando en la distinción operativa entre el espacio de usuario y el espacio de kernel. Asimismo, se busca examinar los protocolos de seguridad basados en firmas digitales y estudiar la gestión de periféricos a través de drivers, comprendiendo los mecanismos de comunicación entre el sistema operativo y el hardware.

Preparación del entorno

siguiendo las instrucciones

```
sudo apt-get update
```

```
sudo apt-get install build-essential checkinstall kernel-package linux-source
```

```
sudo apt-get install linux-headers-$(uname -r)
```

Verificamos la instalación de los headers del kernel

```
ls /lib/modules/$(uname -r)/build
```

Realizando un fork para que nos quede la documentación , hicimos

git clone <https://gitlab.com/francescogallone-group/kernel-modules-sdc.git>

Luego

```
cd kernel-modules-sdc
```

```
ls
```

```
cd part1
```

```
cd module
```

```
cat mimodulo.c
```

```
make
```

```
/* Función que se invoca cuando se carga el módulo en el kernel */
int modulo_lin_init(void)
{
    printk(KERN_INFO "Modulo cargado en el kernel.\n");

    /* Devolver 0 para indicar una carga correcta del módulo */
    return 0;
}

/* Función que se invoca cuando se descarga el módulo del kernel */
void modulo_lin_clean(void)
{
    printk(KERN_INFO "Modulo descargado del kernel.\n");
}

/* Declaración de funciones init y exit */
module_init(modulo_lin_init);
module_exit(modulo_lin_clean);

francesco@Francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ make
make -C /lib/modules/6.17.0-20-generic/build M=/home/francesco/kernel-modules-sdc/part1/module modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: se entra en el directorio '/home/francesco/kernel-modules-sdc/part1/module'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M]  mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
  8 | int modulo_lin_init(void)
    |     ^~~~~~
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
 17 | void modulo_lin_clean(void)
    |     ^~~~~~
MODPOST Module.symvers
CC [M]  mimodulo.mod.o
CC [M]  .module-common.o
LD [M]  mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/francesco/kernel-modules-sdc/part1/module'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-20-generic'
francesco@Francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ ls *.ko
mimodulo.ko
francesco@Francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

```
ls *.ko
```

Esto generó el archivo [mimodulo.ko](#)

Ahora cargamos el módulo al kernel

sudo insmod [mimodulo.ko](#)

Verificamos que cargó

sudo dmesg | tail

```
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ sudo insmod mimodulo.ko
[sudo] contraseña para francesco:
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ sudo dmesg | tail
[ 2137.272230] audit: type=1400 audit(1778857178.136:554): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2142.274199] audit: type=1400 audit(1778857183.137:555): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2147.295774] audit: type=1400 audit(1778857188.158:556): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2152.306807] audit: type=1400 audit(1778857193.169:557): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2157.308856] audit: type=1400 audit(1778857198.171:558): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2159.922915] audit: type=1400 audit(1778857208.785:559): apparmor="DENIED" operation="open" class="file" profile="snap.firmware-updater.firmware-notifier" name="/proc/sys/vm/max_map_
count" pid=28816 comm="firmware-notifi" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2162.510572] audit: type=1400 audit(1778857203.372:560): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[ 2163.398309] mimodulo: module verification failed: signature and/or required key missing - tainting kernel
[ 2163.399212] Modulo cargado en el kernel.
[ 2173.473207] audit: type=1400 audit(1778857214.335:561): apparmor="DENIED" operation="open" class="file" profile="snap.firefox.firefox" name="/proc/pressure/memory" pid=3492 comm="Me
moryPoller" requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

El kernel permite cargar modulos no firmados pero marca el kernel como “tainted” ,
indicando que se ejecuta codigo no verificado

ahora ejecutamos

modinfo [mimodulo.ko](#)

Y además este , para comparar las diferencias con un módulo oficial del kernel

modinfo /lib/modules/\$(uname -r)/kernel/crypto/des_generic.ko

modinfo /lib/modules/\$(uname -r)/kernel/crypto/des_generic.ko.zst

```
moryPoller requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ modinfo mimodulo.ko
filename:      /home/francesco/kernel-modules-sdc/part1/module/mimodulo.ko
author:        Catedra de SdeC
description:    Primer modulo ejemplo
license:        GPL
srcversion:     C6390D617B2101FB1B600A9
depends:
name:          mimodulo
retpoline:     Y
vermagic:       6.17.0-20-generic SMP preempt mod_unload modversions
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

```
name: m1modulo
retpoline: Y
vermagic: 6.17.0-20-generic SMP preempt mod_unload modversions
francesco@francesco-Katana-15-B13VFK: ~/kernel-modules-sdc/part1/module$ modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko
modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
modinfo: ERROR: Module /lib/modules/6.17.0-20-generic/kernel/crypto/des_generic.ko not found.
filename: /lib/modules/6.17.0-20-generic/kernel/crypto/des_generic.ko.zst
alias: crypto-des3_edc-generic
alias: des3_edc-generic
alias: crypto-des3_edc
alias: des3_edc
alias: crypto-des-generic
alias: des-generic
alias: crypto-des
alias: des
author: Dag Arne Osvik <da@osvik.no>
description: DES & Triple DES EDE Cipher Algorithms
license: GPL
srcversion: A4E91C81A384FB1FA6D530F
depends: libdes
intree: Y
name: des_generic
retpoline: Y
vermagic: 6.17.0-20-generic SMP preempt mod_unload modversions
sig_id: PKCS#7
signer: Build time autogenerated kernel key
sig_key: 55:F5:5B:61:38:83:91:B5:25:4F:BC:EB:92:24:4D:CD:0D:E3:1F:8E
sig_hashalgo: sha512
signature: 6A:58:B9:49:69:B4:91:B3:15:06:82:2A:76:46:4E:9B:70:A6:0B:65:
D4:F9:A3:C3:E6:D2:E3:19:35:B3:C8:8D:57:B5:D3:17:D4:12:D3:CB:
EB:16:37:C7:D9:F7:BE:8F:29:43:D1:33:C6:7D:F0:A4:EA:39:87:75:
63:77:E1:B1:71:24:C5:16:06:D8:DE:0E:69:5D:1C:2B:E6:C5:FE:7C:
80:E9:BB:56:84:AB:EC:77:F4:56:9E:2F:90:9D:99:06:22:D5:23:3D:
71:8B:4C:A8:56:92:86:3D:84:01:C3:B5:91:69:04:03:DE:CC:82:AB:
1A:1F:F2:31:D0:1A:07:37:0A:34:82:3E:B8:5C:70:99:3E:60:D5:86:
29:48:B9:81:86:3C:B8:D9:13:34:D8:5D:3A:BE:DF:90:A1:5B:C2:64:
84:36:55:17:20:8E:6F:40:59:56:37:8E:4B:89:53:55:79:59:B4:F1:
8D:86:D3:A6:C1:AC:2E:1D:E3:40:43:81:3B:49:96:AA:B9:04:F0:54:
D6:0D:C9:7E:41:68:0B:E2:2A:4E:61:55:5C:01:B4:4A:7A:B3:5E:9E:
4D:FB:24:96:B9:C6:70:85:38:22:4D:39:EE:68:E4:DE:18:9F:80:EC:
B4:AC:F1:51:EA:BD:C6:71:2A:2D:22:21:05:EE:A7:74:7C:18:E1:55:
A0:32:53:38:09:57:38:AE:6A:9D:6F:AB:E1:06:2B:7E:C6:EF:18:2B:
35:BC:9D:8C:29:89:0A:6E:3E:70:C0:5D:B2:E4:D5:FF:B1:6B:E8:61:
CF:EC:66:C4:04:9A:0B:97:55:BA:AF:B3:8C:B0:80:D1:F3:68:A2:1B:
4A:04:97:AE:C0:63:3F:CB:A5:2D:06:B7:78:9A:13:61:AC:64:F8:11:
B4:EE:0B:CB:8B:B1:F1:D7:68:CC:C7:75:EF:73:7D:E6:59:B2:10:A7:
53:C7:64:3C:58:8D:51:9F:29:5E:09:B9:0C:80:78:EE:14:AC:BA:BC:
1F:73:97:06:09:7B:5F:83:5B:D2:7E:FE:06:5D:7B:85:1E:14:F7:25:
56:D4:E1:2D:78:A3:7A:AC:8C:1F:36:3D:46:E5:31:5F:72:88:CE:C2:
96:5A:B6:F2:99:3F:DB:4A:32:6C:87:EB:B7:82:65:D6:72:95:D1:53:
AB:DD:D1:19:3E:7A:95:29:59:8A:3D:09:88:5A:35:EB:24:4C:31:3E:
4C:1D:ED:AC:FD:73:DD:A2:41:66:64:15:1B:25:06:2F:8C:2B:36:0F:
4E:F5:A9:B9:D2:58:85:AB:3C:AE:48:AA:81:B9:4F:32:36:A9:76:C5:
CE:2B:A0:F4:95:C5:DD:CA:73:08:C2:9B
francesco@francesco-Katana-15-B13VFK: ~/kernel-modules-sdc/part1/module$
```

Desafío 1

Para evitar que el kernel cargue código malicioso, se utiliza la verificación de firmas. Si un módulo no está firmado por una clave de confianza, el kernel lo rechaza.

¿Cómo se evitan los módulos no firmados?

Para forzar esto, se debe configurar el kernel con el parámetro `modulo.sig_enforce=1`. Esto hace que el kernel solo acepte módulos con firma válida, bloqueando intentos de inyectar código no autorizado (como en rootkits).

KSPP (Kernel Self Protection Project): Conjunto de parches para mitigar exploits.

KASLR (Kernel Address Space Layout Randomization): Aleatoriza dónde se carga el kernel en memoria para que los atacantes no sepan dónde "golpear".

Desactivar sysrq: Evita que alguien con acceso físico use combinaciones de teclas para colapsar o reiniciar el sistema.

Restringir dmesg: Evitar que usuarios no privilegiados vean logs del kernel (donde a veces se filtran direcciones de memoria).

¿Qué es un CheckInstall?

Es una herramienta que reemplaza `sudo make install`. En lugar de distribuir archivos por el sistema sin un control, se genera un paquete (.deb, .rpm o Slackware) y se instala usando el gestor de paquetes oficial.

Esto se hace para permitir desinstalar o actualizar software compilado desde código fuente de una forma limpia y organizada.

Ahora probamos un empaquetado de `hello_world`

Primero se instala `checkinstall` y creamos un pequeño código en C

```
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ mkdir ~/hello_test
cd ~/hello_test
francesco@francesco-Katana-15-B13VFK:~/hello_test$ nano hello.c
francesco@francesco-Katana-15-B13VFK:~/hello_test$ gcc hello.c -o hello
francesco@francesco-Katana-15-B13VFK:~/hello_test$ ./hello
Hola mundo
francesco@francesco-Katana-15-B13VFK:~/hello_test$ mkdir -p hello_pkg/usr/local/bin
francesco@francesco-Katana-15-B13VFK:~/hello_test$ cp hello hello_pkg/usr/local/bin/
francesco@francesco-Katana-15-B13VFK:~/hello_test$ cd hello_pkg
francesco@francesco-Katana-15-B13VFK:~/hello_test/hello_pkg$ nano Makefile
francesco@francesco-Katana-15-B13VFK:~/hello_test/hello_pkg$ sudo checkinstall
```

y ahora si en la terminal escribimos `hello`, nos debería devolver un `Hola mundo`

```
francesco@francesco-Katana-15-B13VFK:~/hello_test/hello_pkg$ hello
Hola mundo
francesco@francesco-Katana-15-B13VFK:~/hello_test/hello_pkg$ dpkg-query -f='${Package} ${Version} ${Architecture}\n' -W hello
ii hello-pkg 20260515-1 amd64 Package created with checkinstall 1.6.3
francesco@francesco-Katana-15-B13VFK:~/hello_test/hello_pkg$
```


Desafío 2

¿Qué funciones tiene disponible un programa y un módulo?

La distinción fundamental está en cómo cada entidad accede a los recursos del sistema. Mientras que un programa de usuario depende de bibliotecas externas (como `libc`) y debe solicitar permiso al sistema operativo mediante llamadas al sistema (syscalls) para cualquier tarea crítica, un módulo de kernel opera de forma nativa. Al ejecutarse directamente en el espacio del núcleo, el módulo no requiere intermediarios y utiliza las funciones internas del kernel de manera directa.

Programa	Módulo de kernel
<code>printf()</code>	<code>printk()</code>
<code>malloc()</code>	<code>kmalloc()</code>
<code>free()</code>	<code>kfree()</code>
<code>fopen()</code>	APIs VFS del kernel
<code>read()</code>	<code>copy_from_user()</code>
<code>write()</code>	<code>copy_to_user()</code>
<code>sleep()</code>	<code>msleep()</code>

User Space vs. Kernel Space

Linux implementa una división estricta para garantizar la estabilidad y seguridad del sistema, separando la ejecución en dos entornos jerárquicos.

El Espacio de Usuario (User Space)

Es el entorno restringido donde residen las aplicaciones cotidianas (navegadores, editores de texto, intérpretes como Python). Estos procesos tienen un **acceso limitado al hardware**; no pueden manipular la memoria física o los drivers directamente. Para realizar estas acciones, deben delegar la tarea al kernel a través de una syscall.

El Espacio del Núcleo (Kernel Space)

Acá es donde reside el corazón del SO y los módulos cargados. A diferencia del espacio de usuario, este entorno posee **privilegios totales**, permitiendo la gestión directa de:

Direccionamiento de memoria física.

Controladores de dispositivos (Drivers).

Sistemas de archivos y protocolos de red.

Organización de los Datos y Memoria

Cada programa posee su propio **espacio de datos**, reservado para variables globales y estáticas. Esta región se subdivide típicamente en:

Segmento `.data`: Para variables ya inicializadas con un valor específico.

Segmento `.bss`: Reservado para variables que se inicializan en cero o no tienen valor previo.

Aunque el kernel también posee su propio espacio de datos para gestionar sus variables, la diferencia clave es el **aislamiento**: el espacio de un programa está "enjaulado" por hardware (MMU) para no interferir con otros, mientras que el espacio de datos del kernel es compartido y tiene autoridad sobre todo el mapa de memoria.

Controladores y el Directorio `/dev`

Un **driver** funciona como un traductor especializado entre el hardware y el software. En Linux, muchos se implementan como módulos que pueden insertarse o removerse sin reiniciar.

Algunos ejemplos de dispositivos con drivers pueden ser:

- Teclado

- Mouse
- Disco
- USB
- Audio
- GPU

Directorio `/dev`

Para simplificar la comunicación, Linux expone los dispositivos a través del directorio `/dev` mediante "archivos de dispositivo". Esto permite que un programa interactúe con el hardware usando comandos estándar de lectura y escritura.

Existen dos categorías principales de dispositivos:

Block Devices: Manejan datos en fragmentos o bloques (ej. SSDs, discos rígidos, memorias USB).

Character Devices: Transmiten datos flujo a flujo, byte por byte (ej. terminales, teclados o puertos serie).

En resumen, mientras el driver gestiona el hardware respondiendo a interrupciones, los programas de usuario simplemente leen o escriben en los archivos correspondientes dentro de `/dev`.

Preguntas del trabajo práctico

¿Qué diferencias se pueden observar entre los dos modinfo ?

Campo	mimodulo.ko	des_generic.ko.zst
alias	ninguno	múltiples (des, des3_edc, etc.)
intree	no aparece	Y (es oficial del kernel)
sig_id	no tiene	PKCS#7
signer	no tiene	Build time autogenerated kernel key
sig_hashalgo	no tiene	sha512
signature	no tiene	firma completa
formato	.ko	.ko.zst (comprimido)

La diferencia más importante es que: des_generic está firmado y es intree (oficial), mimodulo no tiene firma y no es oficial. Por eso al cargar mimodulo aparece el warning *module verification failed*.

¿Qué drivers/modulos estan cargados en sus propias pc?

Para poder ver eso debemos ejecutar el comando

```
lsmod
```

Y guardamos la salida en un .txt

```
francesco@francesco-Katana-15-B13VFK:~$ cd tp4_sdc
francesco@francesco-Katana-15-B13VFK:~/tp4_sdc$ lsmod > modulo_francesco.txt
francesco@francesco-Katana-15-B13VFK:~/tp4_sdc$ ls
modulo_francesco.txt
francesco@francesco-Katana-15-B13VFK:~/tp4_sdc$
```

Módulos destacados

Categoría	Módulo	Tamaño (Bytes)	Usado por (Dependencias)	Descripción / Función
Gráficos y Video	xe	~3.87 MB	0 (Carga directa)	El nuevo driver de Intel para arquitecturas de GPU modernas. Uno de los módulos más pesados del sistema.
	i915	~4.83 MB	23	Driver clásico de gráficos integrados Intel, altamente interactivo con el entorno gráfico.
	gpu_sched / ttm	~65 KB / ~126 KB	xe, i915	Administrador de memoria y planificador de tareas para la GPU.
Audio (DSP e Intel)	snd_sof	~401 KB	7 sub-módulos	Sound Open Firmware. Controla el procesador de audio digital inteligente (DSP) de Intel.
	snd_hda_codec_alc269	~139 KB	1	Driver para el chip físico Realtek ALC269 de tu placa madre (auriculares/parlantes).
	soundwire_bus	~1.17 MB	soundwire_intel, etc.	Bus de datos moderno de Intel para interconectar componentes de audio eficientes.
Red y Conectividad	iwlwifi	~647 KB	1 (iwlvm)	Driver maestro de Intel para la placa de red inalámbrica (Wi-Fi).
	mac80211	~1.85 MB	1 (iwlvm)	Pila de software que procesa los protocolos de red inalámbrica de Linux.
	bluetooth	~1.03 MB	34 sub-módulos	Motor central del protocolo Bluetooth del sistema.
Virtualización	kvm_intel / kvm	~569 KB / ~1.44 MB	0 / 1 (kvm_intel)	Habilita el soporte de virtualización nativa por hardware en tu procesador Intel (útil para Docker/VMs).
Almacenamiento	nvme_core / nvme	~233 KB / ~61 KB	3 / 2	Controladores esenciales para la lectura y escritura en tu disco sólido SSD NVMe M.2.
Monitoreo Término	coretemp	~24 KB	0	Lee los sensores térmicos internos de los núcleos de tu CPU para evitar sobrecalentamientos.

Cada integrante guarda su salida y se compara con *diff*

```
lsmod > tp4_sdc/lsmod_<modulos_nombre+apellido>.txt
```

```
diff -u modulo_francesco.txt modulos_EzequielMarredo.txt > comparacion_modulos.txt
```

¿Cuales módulos no están cargados pero están disponibles?

Los módulos disponibles se encuentran en `/lib/modules/$(uname -r)/kernel/`. Se pueden ver las principales categorías:

```
ls /lib/modules/$(uname -r)/kernel/
```

Para contar el total de módulos disponibles:

```
find /lib/modules/$(uname -r)/kernel/ -name "*.ko*" | wc -l
```

Módulos cargados actualmente

```
lsmod | wc -l
```

```
francesco@francesco-Katana-15-B13VFK:~$ ls /lib/modules/$(uname -r)/kernel/
arch  crypto  fs      kernel  mm      samples  ubuntu      virt
block drivers io_uring lib     net     sound    v4l2loopback  zfs
francesco@francesco-Katana-15-B13VFK:~$ find /lib/modules/$(uname -r)/kernel/ -name "*.ko*" | wc -l
6767
francesco@francesco-Katana-15-B13VFK:~$ lsmod | wc -l
195
francesco@francesco-Katana-15-B13VFK:~$
```

Esto nos enseña el diseño modular del kernel, solo se carga en la memoria lo estrictamente necesario para el hardware y funcionalidades en uso.

¿Qué pasa cuando el driver de un dispositivo no está disponible?

Hay tres posibilidades:

El módulo no existe para ese hardware -> el dispositivo no funciona. No aparece en `/dev` y el sistema no lo reconoce. Un ejemplo típico es conectar un adaptador WiFi sin driver disponible: no aparece ninguna interfaz de red.

El módulo existe, pero no está cargado -> el kernel puede cargarlo automáticamente al detectar el dispositivo mediante ***modprobe***. El dispositivo funciona normalmente.

El módulo existe pero falla al cargar-> el sistema bloqueará la carga arrojando un error, lo que impedirá la habilitación del hardware. En escenarios más críticos, este fallo de ejecución puede desencadenar un *kernel panic*, comprometiendo la estabilidad de todo el sistema operativo.

Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte

Luego de ejecutar el comando

`hwinfo --short` (nos genera un resumen del hardware) con `|` mandamos la salida al comando `nc termbin.com 9999` o sea se usa *netcat* para subir el texto a termbin y nos devuelve una url publica temporal.

```
francesco@francesco-Katana-15-B13VFK:~$ hwinfo --short | nc termbin.com 9999
https://termbin.com/0cp4
francesco@francesco-Katana-15-B13VFK:~$
```

El link es

<https://termbin.com/Ocp4>

¿Qué diferencia existe entre un módulo y un programa?

La diferencia clave entre un programa común y un módulo de kernel está en dónde corren, qué permisos tienen y qué pasa si fallan. Un programa funciona en el Espacio de Usuario (**User Space**), lo que significa que tiene permisos limitados y no puede tocar el hardware directamente; si necesita hacer algo importante, tiene que pedirle permiso al sistema operativo a través de funciones y llamadas al sistema.

En cambio, un módulo se mete a trabajar directamente dentro del Espacio de Kernel (**Kernel Space**), volviéndose parte del motor del sistema, por lo que tiene acceso total y directo a la memoria y a los componentes físicos de la computadora. Esta diferencia es vital cuando ocurren errores: si un programa anda mal o se tilda, el sistema operativo lo cierra a la fuerza y la computadora sigue andando como si nada. Pero si un módulo del kernel tiene un error de código, como comparte la memoria principal con todo el corazón del sistema, no hay ninguna red de seguridad que lo frene; el error arrastra a todo lo demás y provoca que la pantalla se congele por completo o que la máquina tire un **kernel panic!!!!** y se reinicie.

¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple helloworld en c?

Según la bibliografía para ver esto podemos utilizar una herramienta llamada **Strace**, que intercepta y muestra todas las llamadas al sistema que el programa le realizará al kernel. Utilizando el helloworld creado en el desafío 1 para mostrar la salida impresa.

Firmado de un módulo

```
francesco@francesco-katana-15-B13VFK: /kernel-modules-sdc/parti/module$ openssl req -new -x509 -newkey rsa:2048 \
-keyout MOK.priv \
-outform DER \
-out MOK.der \
-nodes \
-days 36500 \
-subj "/CN=Francesco Kernel Module/"
```

Perform MOK management

Continue boot
Enroll MOK
Enroll key from disk
Enroll hash from disk

```
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ ls
description-pak  Makefile  mimodulo.c  mimodulo.ko  mimodulo.mod  mimodulo.mod.c  mimodulo.mod.o  modules.order  Module.symvers  MOK.der  MOK.priv
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ find /usr/src -name sign-file
/usr/src/linux-headers-6.17.0-23-generic/scripts/sign-file
/usr/src/linux-headers-6.17.0-29-generic/scripts/sign-file
/usr/src/linux-headers-6.17.0-20-generic/scripts/sign-file
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ uname -r
6.17.0-23-generic
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ /usr/src/linux-headers-6.17.0-23-generic/scripts/sign-file \
sha256 \
./MOK.priv \
./MOK.der \
mimodulo.ko
```

Pág 17

```
cd ~/kernel-modules-sdc/part1/module
modinfo mimodulo.ko | tail -n 20
```

```
francesco@francesco-Katana-15-B13VFK:~$ cd ~/kernel-modules-sdc/part1/module
modinfo mimodulo.ko | tail -n 20
name:                mimodulo
retpoline:           Y
vermagic:             6.17.0-20-generic SMP preempt mod_unload modversions
sig_id:               PKCS#7
signer:               Francesco Kernel Module
sig_key:              3A:81:90:0C:EE:AD:25:9F:6E:30:A1:31:8A:E1:9C:FE:EE:BA:DA:31
sig_hashalgo:         sha256
signature:            36:B7:C7:C5:F6:89:56:A6:ED:C2:B7:21:AC:06:D8:8E:19:52:69:E1:
                     47:59:09:E1:4D:C5:B2:67:8F:B9:47:CE:C1:22:48:D3:49:61:60:99:
                     1F:F3:8A:E0:D7:FE:51:79:82:67:60:3A:5F:AD:B4:CE:D6:13:FF:90:
                     BF:A8:9A:C6:4E:34:58:D0:CA:5C:71:2F:E1:9C:E0:DC:03:A3:8C:E7:
                     32:1D:5A:E3:45:D0:A0:5C:86:08:42:47:78:51:37:CD:42:29:32:59:
                     3B:45:F4:DC:CD:A4:88:61:7E:4F:FD:54:B0:50:57:0F:36:25:E0:90:
                     F8:F2:ED:49:5F:90:53:4B:E5:E3:25:DC:3C:C4:BB:1A:7E:81:01:B4:
                     E1:52:F1:52:DA:06:E5:B9:78:88:05:83:80:7F:EF:23:80:8C:3B:6D:
                     E2:5A:F1:87:59:D9:ED:23:CD:2C:EF:93:22:BD:BF:5D:AC:22:15:8A:
                     C3:B4:F1:B9:A7:1B:DA:2C:7C:29:6F:B5:E8:66:62:E5:5E:AA:5A:0B:
                     6B:A1:83:CD:B0:F3:0F:19:CC:A7:47:08:16:09:D3:FB:B7:54:62:97:
                     3E:5C:CD:ED:11:17:11:CA:41:95:86:C5:BE:13:66:B9:A9:8E:E8:D6:
                     99:37:7A:B8:20:DB:7F:F2:A1:FC:75:D9:6F:ED:5C:1A
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

Agregar evidencia de la compilación, carga y descarga de su propio módulo imprimiendo el nombre del equipo en los registros del kernel.

Primero se compila el módulo utilizando make.

```
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$ make
make -C /lib/modules/6.17.0-29-generic/build M=/home/francesco/kernel-modules-sdc/part1/module modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-29-generic'
make[2]: se entra en el directorio '/home/francesco/kernel-modules-sdc/part1/module'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M]  mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
   8 | int modulo_lin_init(void)
     |             ^
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
  17 | void modulo_lin_clean(void)
     |             ^
MODPOST Module.symvers
CC [M]  mimodulo.mod.o
CC [M]  .module-common.o
LD [M]  mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/francesco/kernel-modules-sdc/part1/module'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-29-generic'
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

Sistemas de Computación - Mr.Robot

Luego , se carga el módulo con las siguientes instrucciones.

```
sudo insmod mimodulo.ko
```

```
sudo dmesg
```

```
lsmod | grep mod
```

```
[ 2423.231061] Modulo cargado en el kernel - Equipo: francesco-Katana-15-B13VFK
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

Se realiza la descarga del módulo y se verifica si se realiza correctamente.

```
sudo rmmod mimodulo
```

```
sudo dmesg | grep "Modulo"
```

```
[ 2423.231061] Modulo cargado en el kernel - Equipo: francesco-Katana-15-B13VFK
[ 2571.975543] Modulo descargado del kernel - Equipo: francesco-Katana-15-B13VFK
francesco@francesco-Katana-15-B13VFK:~/kernel-modules-sdc/part1/module$
```

¿Que pasa si mi compañero con secure boot habilitado intenta cargar un módulo firmado por mi?

El sistema rechazará el módulo y no se cargará, devolviendo

```
Required key not available
```

Esto ocurre porque aunque el módulo esté firmado, el kernel de mi compañero no tiene mi llave pública en su lista de llaves confiables. Para Secure Boot , una firma de un autor desconocido (yo) es igual de inválida que no tener firma.

¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

La consecuencia principal fue que **muchos sistemas con dual boot dejaron de poder iniciar Linux**. Un parche de seguridad de Microsoft para corregir una vulnerabilidad de GRUB (**CVE-2022-2601**) aplicó una política **SBAT (Secure Boot Advanced Targeting)** que terminó bloqueando algunos cargadores de arranque (shim/GRUB) de Linux, aun en equipos con Windows + Linux, donde en teoría no debía aplicarse. Los usuarios recibían errores como:

“Something has gone seriously wrong” y “Security Policy Violation”

y linux no arrancaba cuando **Secure boot** estaba activado.

¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?

Desactivar Secure Boot permite volver a iniciar Linux y evita el bloqueo del cargador de arranque, pero implica reducir la seguridad del proceso de boot. Al estar deshabilitado, el sistema deja de verificar criptográficamente que el bootloader, kernel y componentes de arranque sean confiables, lo que aumenta el riesgo de ejecutar firmware o software malicioso durante el arranque (por ejemplo, bootkits o rootkits persistentes). Es una solución práctica, pero sacrifica parte de la protección del sistema

¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?

El propósito principal de Secure Boot es garantizar la integridad y autenticidad del proceso de arranque, permitiendo que solo se ejecuten bootloaders, kernels y componentes firmados y confiables. Su objetivo es impedir que malware, rootkits o firmware alterado se carguen antes del sistema operativo, protegiendo el equipo desde las primeras etapas del boot.